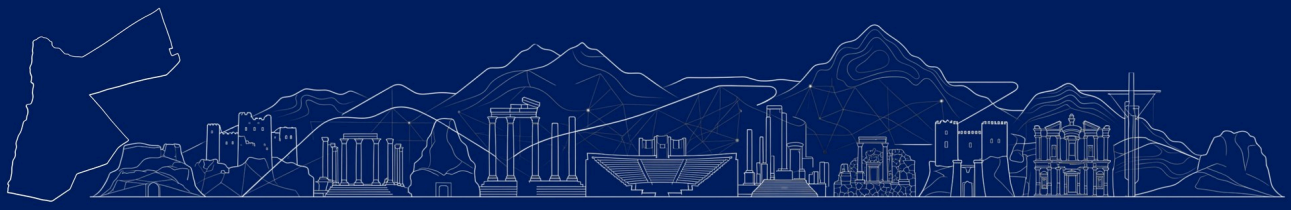
AI FUTURE 26

YOUTH EDITION

42 IRBID | M.A.T ACADEMY



AI · CYBERSECURITY · DRONES · ROBOTICS

THE FUTURE OF SMART CITIES

AI HACKATHON · CHALLENGE BRIEF

Government Services Chatbot

Build a chatbot that answers citizens' questions about Jordan's passport, family book and ID card services, correctly, and only from the official documents.

دائرة الأحوال المدنية والجوازات

DATA	MODEL	TIME	DELIVER
12 service documents, Arabic	**Same ~7B model for all, via Ollama**	**4 hours, teams of 3–4**	**A `main.py` we can run**

1 The challenge

Citizens keep asking the same practical questions: how much does it cost, which documents do I bring, who can apply, what if I lost it? The answers exist in official service pages, but those pages are messy Arabic text full of conditions and exceptions.

Your job: build a chatbot that answers these questions **correctly**, **only from the documents we give you**, using the small model we provide through Ollama. Any approach and any tools are allowed.

2 The data

You get 12 text files in `data/`, one per service. They cover three official documents, each with four situations:

	FIRST TIME	RENEWAL	LOST	DAMAGED
Passport جواز السفر	جواز سفر لأول مرة `passport_first_issue`	تجديد جواز السفر `passport_renewal`	جواز سفر بدل فاقد `passport_lost`	جواز سفر بدل تالف `passport_damaged`
Family book دفتر العائلة	دفتر عائلة لأول مرة `family_book_first_issue`	تجديد دفتر العائلة `family_book_renewal`	دفتر عائلة بدل فاقد `family_book_lost`	دفتر عائلة بدل تالف `family_book_damaged`
Smart ID البطاقة الذكية	بطاقة ذكية لأول مرة `id_card_first_issue`	تجديد البطاقة الذكية `id_card_renewal`	بطاقة ذكية بدل فاقد `id_card_lost`	بطاقة ذكية بدل تالف `id_card_damaged`

The grey name under each service is its file name in `data/`.

What a file looks like

Every file has the same nine fields: who it's for (الفئة المستفيدة), where to apply (أماكن تقديم الخدمة), conditions (الشروط), required documents (الوثائق المطلوبة), procedure (الإجراءات), time (الزمن المستغرق), partner agencies (شركاء الخدمة), notes (ملاحظات) and fees (الرسوم). The text is raw, straight from the website. The highlighted lines are leftover website menus:

```
الرئيسية
الخدمات
دليل الخدمات
جواز سفر بدل تالف
شارك
جواز سفر بدل تالف
الفئة المستفيدة  المواطنون الأردنيون
...
الرسوم  100 دينار
```

3 What makes it hard

Look-alike services

The four passport files share most of their text. So do the family book and ID card files. Often only one fee, one document or one note is different. If your bot picks the wrong file, its answer looks perfect and is still wrong.

Fees depend on the person

Many fees are not one number: they change with age, whether the document has expired, inside or outside Jordan, how many times it was lost. Users describe their situation; the bot must apply the right rule.

Facts hide in "notes"

Extra charges, deadlines, waiting periods and exceptions are often under ملاحظات, not under fees or conditions.

People write in dialect

Users say الباسبور, الهوية, اهترى, شو بلزمني؟, قديش بدفع؟, while the files say جواز السفر, البطاقة الذكية, بدل تالف, الرسوم, الوثائق المطلوبة.

The data is messy

Website menu lines, tabs and blank lines, three numbering styles, typos, repeated lines, copy-paste leftovers, and amounts written as digits in one place and as words in another. Clean it any way you want, but don't change the facts.

A small model has limits

A 7B model makes reasoning mistakes and may "remember" outdated facts from training. Ollama's default context window is small, and text beyond it can be cut off without any error. Answers must come from the documents.

Knowing when not to answer

Some questions are not covered by the 12 files, including services that sound related but aren't in your data. A good bot says it doesn't have that information instead of inventing an answer. When a question is unclear, a good bot asks what the user means.

4 Questions you will face

The real test questions are hidden. They cover all 12 services and these types. The examples below are **not** in the test:

Type	Example	A good answer...
Simple fact	كم يستغرق إصدار دفتر العائلة؟	gives the exact value
Fee for a situation	عمري 17 وجوازي باقي عليه 3 شهور، كم رسوم التجديد؟	applies the right rule
Documents / conditions	شو المطلوب لتجديد دفتر العائلة؟	gives the full list for that service
Hidden in notes	إذا توفي الأب والأم، مين بصير رب الأسرة؟	finds the rule in the notes
Comparison	أيهما أغلى: جواز بدل تالف ولا بدل فاقد؟	uses both services correctly
Dialect	الباسبور خلص، شو بعمل؟	understands it, answers clearly

Conversation (bonus)	A follow-up such as وكم بتكلف؟ after a first question	remembers what the chat is about
Long story	A long message about several people or problems, with details that don't matter	answers every part that was asked
Wrong assumption	الزوجة لازم تجيب موافقة جوزها عشان تطلع جواز، صح؟	corrects it from the document
Unclear	بدي أجدد، شو المطلوب؟	asks which document
Not in the data	كم رسوم تصديق الشهادة الجامعية؟	says it doesn't know; invents nothing

5 What we will run

Build your chatbot any way you like. For the evaluation we only need one thing: **a `main.py` that reads a questions file and writes an answers file.**

1 **Your project folder has a `main.py`. We put our `questions.json` next to it and run:**

```
$ ls
data/   main.py   questions.json   # + any other files and folders your project needs

$ python main.py questions.json
```

`main.py` is only the **entry point**. Organize the rest of your project however you like (other modules, cleaned data, indexes, config files).

2 **When it finishes, a new file `answers.json` is in the same folder. That file is what we score.**

```
$ ls
answers.json   data/   main.py   questions.json   # + your other files
```

questions.json: what you receive

A list of **chats**. Each chat is one user talking to your bot and has a list of **messages**. Most chats have **one** message, a normal question. Some chats have **several**: a conversation.

```
{
  "chats": [
    {
      "chat_id": "chat-01",
      "messages": [
        {"message_id": "chat-01-1", "question": "كم يستغرق تجديد دفتر العائلة؟"}
      ]
    },
    {
      "chat_id": "chat-02",
      "messages": [
        {"message_id": "chat-02-1", "question": "شو الأوراق المطلوبة لتجديد دفتر العائلة؟"},
        {"message_id": "chat-02-2", "question": "وبقدر أقدّم الطلب من السفارة؟"}
      ]
    }
  ]
}
```

answers.json: what you write

The same file, with two keys added to every message: `answer` and `contexts`. Keep everything else as it is.

```
$ cat answers.json
{
  "chats": [
    {
      "chat_id": "chat-01",
      "messages": [
        {"message_id": "chat-01-1",
         "question": "كم يستغرق تجديد دفتر العائلة؟",
         "answer":   "يستغرق تجديد دفتر العائلة 45 دقيقة.",
         "contexts": ["تجديد دفتر العائلة - الزمن المستغرق: 45 دقيقة"]}
      ]
    },
    …
  ]
}
```

Key	What to put
`answer`	Your chatbot's answer: the text a user would read.
`contexts`	A list of the pieces of text your bot used to write this answer, taken from your data (cleaned or not). Only the pieces you used, **not whole files** and not a summary. `[]` if it found nothing.

Chats with several messages (bonus)

- Answer the messages of a chat **in order**. A later message can depend on earlier ones: in `chat-02` above, "وبقدر أقدّم الطلب من السفارة؟" means the *family book renewal* from the message before.
- Every chat is a new user: forget everything from the previous chat.
- These count as **bonus**: they add points and never lower your main score. No time for memory? Answer each message on its own; you only miss part of the bonus.

Our scoring is automatic, so the format must be exact

- **Copy the structure.** Keep `chats`, `chat_id`, `messages`, `message_id` and `question` exactly as you received them, and add `answer` and `contexts` to every message. You may add other keys of your own; we ignore them.
- Answer every message, even if your bot fails on it. A missing answer scores zero.
- The file must be named `answers.json`, in the same folder, valid JSON, saved as UTF-8.
- Your program must work for **any number** of chats and messages. The real file is much bigger than the practice one.

Common mistakes to avoid

Mistake	What to do
Crash on Windows when reading or writing Arabic text	Always open files with `encoding="utf-8"`, e.g. `open("answers.json", "w", encoding="utf-8")`.
Works with internet, fails in the evaluation	Before the run, download everything: `ollama pull <model>`, any embedding models, pip packages. Then turn Wi-Fi off and test once.
Program needs a virtual environment	Tell us the exact command, e.g. `.venv/bin/python main.py` or `venv\Scripts\python main.py`.
One error stops the whole run and no file is written	Catch errors per message and still write an answer, e.g. `"answer": "error"`. A partial file scores much better than none.
Very slow: each answer takes minutes	The real file has many questions. Keep prompts and answers short, check Ollama's context size, and time your program on the practice file.
Answers in the wrong place	Write answers into the same chat and message they belong to. Never reorder or merge chats.

Test it yourself

You get `questions.json` (5 chats, one with 2 messages) and `answers_example.json` with correct answers for them. Run `python main.py questions.json` and compare your `answers.json` with the example: **the structure must be the same**. Your wording can differ.

6 How you are evaluated

Every team gets the same hidden questions. Each answer is compared with a reference answer written from the documents.

Criterion	What we look for
Correct answer	The right amounts, documents and conditions for the user's situation, with nothing wrong added. Out-of-scope questions are declined; unclear ones are clarified. This matters most.
Right source	The `contexts` you return come from the service(s) the question is about. We recognize this from the text itself, cleaned or not.
Grounded answer	What the answer says can be found in your `contexts`. An answer from the model's memory counts as unsupported, even if it's right.

Answer the question that was asked. Pasting a whole document is not an answer. You'll also give a short walkthrough of your solution: how you handled the data, how your bot finds the right information, and how you stop it from making things up.

Rules

- Use only the provided files as the knowledge source.
- Every answer must be generated by the provided model when your program runs. No pre-written answers, and no other AI models or online AI services.
- You may use any tools while building, including AI coding assistants, and you may clean and restructure the data, but not change its facts. Local helper models (embeddings, rerankers) are allowed.
- We run `python main.py questions.json` on your laptop **with the internet off**. Download the model and every package beforehand. If your program needs a different command, tell us before the run.